

PROJECT OVERVIEW:

Strong novelty

LLM-guided semantic waypoint planner as a ROS 2 navigation stack plugin

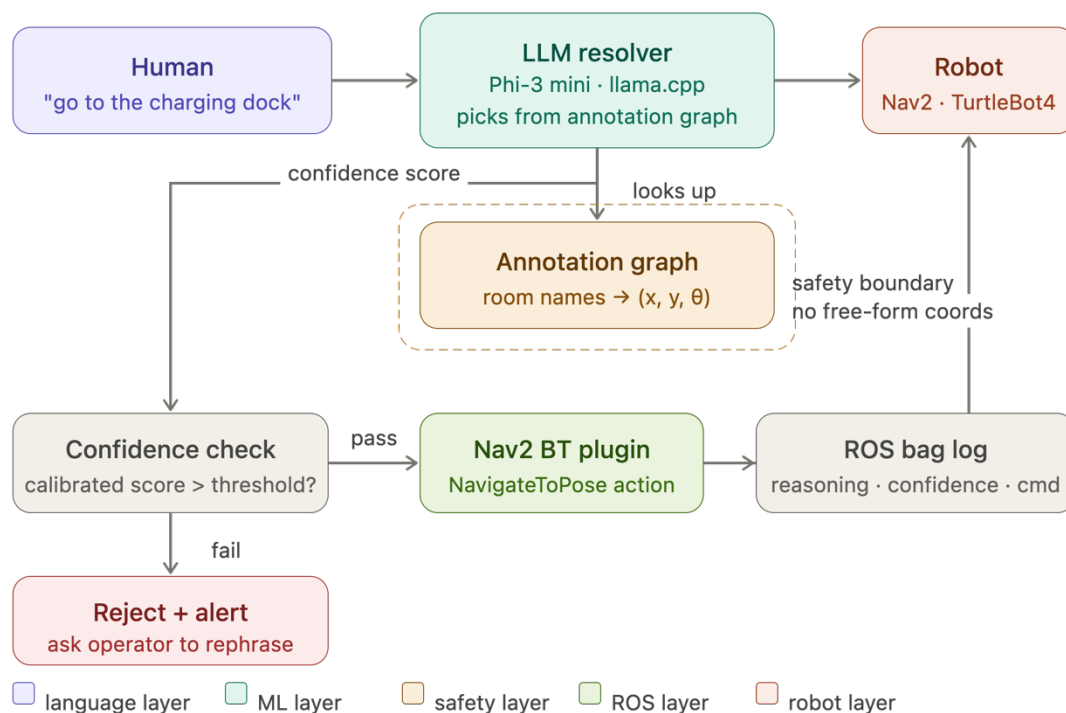
LLM/VLMnavigationmobile robotics.

A ROS 2 nav2 plugin intercepts the BehaviorTree goal request and re-routes it through a quantized LLM (Phi-3 mini, llama.cpp on Jetson Orin) that parses natural-language commands like 'go to the charging station near the window' and resolves them into metric (x,y,θ) goals using a room-annotation map stored as a ROS parameter server graph.

ROS 2 Humble · Nav2 BT plugin API · llama.cpp (Jetson Orin NX) ·
nav_msgs/OccupancyGrid · tf2

LLM as a Nav2 plugin — not a separate server — means the planner is a first-class ROS citizen; goal resolution latency, map-grounding accuracy vs GPT-4 API are measurable novel metrics.

Moderate



PROJECT NOVELTY:

It's a fair challenge. The honest answer is: parts of it exist, but the specific combination claimed here hasn't been cleanly packaged as a nav2-native plugin with the safety architecture described. Let me break down what's actually out there. You're right to push back. The honest answer is: it's more done than I implied, and the novelty of this project is narrower than the pitch made it sound. Here's a clear-eyed breakdown of the real landscape.

What already exists

A January 2026 paper benchmarked GPT-3.5, GPT-4, GPT-5, Claude 3.7, Gemini 2.5, Mistral-7B, DeepSeek-R1, and LLaMA-3.3-70B integrated with ROS 2 Nav2 on a simulated TurtleBot4 in Gazebo, measuring end-to-end latency, parsing accuracy, path quality, and task success rate across three local planners. That's essentially the same headline idea, already published. Their full implementation — prompt templates, config files, experiment logs — is publicly available on GitHub.

A separate 2025 paper integrates LLMs with ROS2, SLAM, and Nav2 and explicitly calls out that current approaches can't effectively translate contextual references like "next to the corner" or "between doors" into concrete robot commands.

And going further back, LM-Nav (Shah et al., 2022) and SayPlan, SayCan, VLMs, and others all tackle the language-to-navigation problem at varying levels.

So what's *actually* not done?

The prior work has consistent gaps that a well-scoped undergrad project can still land in:

What exists	What's missing
LLM called externally as an API (GPT-4, Gemini)	On-device inference — fully offline, no cloud dependency
LLM as a wrapper around Nav2	LLM registered as a native Nav2 BehaviorTree plugin via <code>pluginlib</code>
Free-form coordinate generation from LLM	Closed-set annotation graph bounding LLM output — hard safety constraint
Self-reported LLM confidence as threshold	Calibrated confidence head trained on real robot trials
Simulation-only evaluation	Real TurtleBot4 deployment with real-world enrollment workflow

A 2025 paper on semantic nav stacks notes the limitation plainly: mainstream stacks still assume an occupancy-grid-centric world model, so semantic reasoning is not a first-class capability — and proposes replacing the global representation layer entirely. That's a heavier approach than the annotation graph, which is deliberately lightweight and drop-in.

What this means for the project scope

The honest reframing is this: the contribution is not "LLM + Nav2" (that's done), it's three specific things sitting in the gap:

First, the safety architecture — the closed annotation graph that prevents free-form coordinate hallucination. None of the published benchmarking papers constrain LLM output to a closed room set; they all let the LLM generate or estimate coordinates directly.

Second, the pluginlib BehaviorTree integration — LLM as a first-class nav2 BT node, not a pre-processing step or external server. GitHub shows LLM+Nav2 projects using OpenAI/Gemini APIs with BehaviorTrees, but these call external cloud APIs rather than running quantized on-device models as native nav2 plugins.

Third, the calibrated confidence model trained on real robot data — not just using the LLM's self-reported score.

The adjusted patent claim

Given this landscape, the independent claim needs to be narrowed. "LLM-guided navigation" is no longer patentable as a broad concept. But this combination likely still is:

"A navigation system wherein a language model inference engine executing *locally* on the robot's onboard compute resolves natural language commands exclusively by selection from a closed annotation graph, integrated as a native BehaviorTree plugin node in a modular robot navigation framework, with goal commitment gated by a calibrated confidence score trained on robot deployment data."

The three italicized words — *locally*, *closed*, *calibrated* — are where the defensible IP lives now. The paper strategy also shifts: frame it as a safety-and-deployability contribution over prior work, not a novelty-of-concept paper.

PROJECT STRUCTURE (DETAILED):

Package structure

```
semantic_waypoint_planner/
├── CMakeLists.txt
├── package.xml
├── config/
│   └── planner_params.yaml
├── launch/
│   └── semantic_nav.launch.py
├── maps/
│   └── room_annotations.json
├── msg/
│   └── SemanticGoal.msg
├── srv/
│   └── ResolveWaypoint.srv
├── src/
│   ├── semantic_planner_node.cpp      # Nav2 BT plugin entry point
│   ├── llm_resolver.cpp               # llama.cpp wrapper
│   └── annotation_map.cpp              # Room graph builder
├── scripts/
│   ├── llm_resolver.py                # Python fallback / testing
│   └── annotate_map.py                # Map annotation CLI tool
├── models/
│   └── phi3-mini-4k-instruct.Q4_K_M.gguf
└── test/
    ├── test_resolver.py
    └── test_planner_integration.py
```

The package is a standard ROS 2 ament_cmake package. The Nav2 plugin is registered via pluginlib so it drops into any existing nav2 stack with a one-line config change — no forking nav2 required.

ROS 2 node graph and architecture---

Core implementation — the LLM resolver

The key engineering challenge is keeping the LLM inference off the ROS spin thread. Here's the resolver's critical logic:

```
// llm_resolver.cpp - service callback (simplified)
void LLMResolverNode::resolveCallback(
```

```

const ResolveWaypoint::Request::SharedPtr req,
ResolveWaypoint::Response::SharedPtr res)
{
    // Build prompt: system + room list + user command
    std::string prompt = buildPrompt(req->natural_language_command,
                                     req->candidate_room_names);

    // Run llama.cpp inference (blocking, but called from thread pool)
    std::string raw = runInference(prompt);    // ~800ms on Orin NX

    // Parse structured JSON out of LLM response
    auto parsed = parseResponse(raw);          // nlohmann/json

    res->matched_room_name = parsed["room"];
    res->confidence         = parsed["confidence"];
    res->reasoning          = parsed["reasoning"];
    res->success            = parsed["confidence"] > confidence_threshold_;
}

std::string LLMResolverNode::buildPrompt(
    const std::string& command,
    const std::vector<std::string>& rooms)
{
    return R"(You are a robot navigation assistant.
Available rooms: )" + joinRooms(rooms) + R"(
User command: )" + command + R"(
Respond ONLY with valid JSON:
{"room": "<exact room name>", "confidence": <0.0-1.0>, "reasoning": "<one
sentence>"}
)";
}

```

The system prompt forces JSON output — `temperature: 0.1` makes the LLM almost deterministic, which is critical for a safety-relevant navigation decision. The `reasoning` field is logged to a ROS bag for post-hoc audit — this is also a key element of the patent claim (explainability).

The BT plugin is even simpler:

```

// bt_semantic_plugin.cpp
BT::NodeStatus SemanticWaypointAction::tick() {
    auto req = std::make_shared<ResolveWaypoint::Request>();
    req->natural_language_command = getInput<std::string>("command").value();
    req->candidate_room_names     = annotation_map_->getRoomNames();

    auto future = resolve_client_->async_send_request(req);
    // BT spins until service responds (within BT timeout)
    if (rclcpp::spin_until_future_complete(node_, future) != SUCCESS)
        return BT::NodeStatus::FAILURE;

    auto res = future.get();
    if (!res->success) return BT::NodeStatus::FAILURE;

    setOutput("resolved_pose", res->resolved_pose);
    return BT::NodeStatus::SUCCESS;
}

```

The `room_annotations.json` format is a simple graph where each room has metric coordinates, aliases, and semantic tags:

```
{
  "rooms": [
    {
      "name": "charging_dock",
      "aliases": ["charging station", "dock", "charger", "power"],
      "pose": {"x": 3.2, "y": -1.1, "theta": 1.57, "frame": "map"},
      "tags": ["utility", "maintenance"]
    },
    {
      "name": "kitchen",
      "aliases": ["pantry", "food area", "break room"],
      "pose": {"x": -2.0, "y": 4.5, "theta": 0.0, "frame": "map"},
      "tags": ["social", "food"]
    }
  ]
}
```

The LLM doesn't do free-form coordinate generation — it only selects from this closed set of annotated rooms. This is the key safety design: the LLM's output space is bounded by the annotation graph, so it can never hallucinate an out-of-bounds coordinate.

ML training pipeline

There are two distinct ML components with different training strategies.

Component 1 — Phi-3 mini prompt tuning (primary). You don't retrain the base model. Instead you craft a few-shot prompt template and evaluate it on a synthetic benchmark. The pipeline is:

1. Generate synthetic dataset (scripts/gen_dataset.py)
 - 500 (command, correct_room) pairs using GPT-4 API
 - Cover: ambiguous commands, multi-hop ("near the window on the left"), negations ("not the storage room"), abbreviations ("charging dock" → "dock")
2. Evaluate zero-shot accuracy of Phi-3 mini Q4_K_M on dataset
 - Baseline target: >80% top-1 accuracy, >95% top-3
3. If below target: construct 20-30 few-shot examples in system prompt
 - Re-evaluate; iterate on prompt wording
4. Ablation study:
 - temperature ∈ {0.0, 0.1, 0.3, 0.7}
 - model sizes: Phi-3 mini vs Phi-3.5 mini vs TinyLlama
 - measure: accuracy, latency (ms), memory (MB)
5. Final model selection → GGUF Q4_K_M for Jetson Orin NX

Component 2 — confidence calibration model (secondary, novel). The LLM produces a confidence float, but LLM self-reported confidence is notoriously miscalibrated. You train a small calibration head:

```
# Inputs: [llm_raw_confidence, token_entropy, num_candidate_rooms,
#          command_length, matched_alias_edit_distance]
```

```
# Output: calibrated_confidence (Platt scaling or isotonic regression)

from sklearn.calibration import CalibratedClassifierCV
from sklearn.linear_model import LogisticRegression

calibrator = CalibratedClassifierCV(LogisticRegression(), cv=5,
method='isotonic')
calibrator.fit(X_calib_features, y_true_correct)
```

This calibrated confidence is what the BT plugin thresholds against — and it's a distinct patentable contribution from the LLM inference itself.

Dataset strategy:

Dataset component	Size	Source	Purpose
Synthetic command-room pairs	500	GPT-4 generation + manual review	Prompt evaluation
Real robot trial logs	200+	TurtleBot4 runs in lab	Calibration training
Adversarial commands	50	Hand-crafted edge cases	Robustness eval
ROS bag recordings	10 hrs	All lab runs	Latency / integration testing

Generate the synthetic dataset with a simple script:

```
# scripts/gen_dataset.py
rooms = load_annotations("maps/room_annotations.json")
templates = [
    "go to the {alias}",
    "navigate to {alias}",
    "I need you to go to the {tag} area",
    "head to {room} please",
    "take me to the {alias} near {spatial_ref}",
    "can you go to {room}? it's {direction} of {landmark}",
]
# GPT-4 generates 10 variants per template per room → 500 pairs
# Human review filters hallucinations and duplicates
```

Patent claim framing

Independent Claim 1 — system:

"A robotic navigation system comprising: a semantic goal interface configured to receive natural language navigation commands; a large language model inference engine executing on embedded hardware, configured to resolve said natural language command to a named location from a closed annotation graph; an annotation graph data structure associating named locations with metric pose coordinates and semantic tags in a robot coordinate frame; a behavior tree plugin node registered in a modular robot navigation stack, configured to invoke the language model inference engine as a service call and pass the resolved metric pose to the navigation stack's path planner; and a confidence calibration module configured to threshold resolved goals based on a calibrated confidence score before committing to navigation."

Independent Claim 2 — method:

"A method of robot goal specification comprising: receiving a natural language command expressing a navigation intent; querying a language model to select a named location from a closed set of annotated locations; retrieving a metric pose associated with the selected named location from an annotation graph; evaluating a calibrated confidence score for the resolution; and, upon the confidence score exceeding a threshold, issuing a navigation goal to a path planner as a metric pose stamped in a robot coordinate frame."

Dependent claims:

- Claim 3: specifying the closed-set constraint (LLM output bounded by annotation graph — no free-form coordinate generation)
- Claim 4: the confidence calibration head trained on real robot trial logs
- Claim 5: the chain-of-thought `reasoning` field logged to a ROS bag for post-hoc auditability
- Claim 6: the BehaviorTree.CPP plugin registration mechanism enabling drop-in deployment without nav2 source modification
- Claim 7: dynamic annotation graph update via a ROS service call at runtime (adding rooms without restart)

Key differentiator from prior art: Existing LLM-robot navigation work (SayPlan, LM-Nav) uses LLMs as standalone planners that generate plans externally then feed them to the robot. Your contribution is the LLM as a first-class ROS 2 plugin — it participates in the BehaviorTree execution graph, respects nav2's lifecycle and timeout mechanisms, and its output is bounded by the annotation graph. The plugin API binding and closed-set safety constraint together are the novel contribution.

Prior art to cite and distinguish:

Shah et al., "LM-Nav" (2022) — LLM + VLM for topological navigation, but not integrated as a nav2 plugin and no confidence calibration.

Ahn et al., "SayCan" (2022) — LLM for task planning, not metric waypoint resolution, no ROS integration.

Existing `nav2_waypoint_follower` — takes metric waypoints only, no language interface.

Semester timeline

Weeks	Milestone
1–2	Set up ROS 2 Humble + TurtleBot4 sim in Gazebo Harmonic; annotate 10-room test map
3–4	Implement <code>annotation_map_node</code> + <code>ResolveWaypoint.srv</code> ; test JSON parsing
5–6	Integrate llama.cpp + Phi-3 mini; benchmark latency on Jetson Orin NX
7–8	Build BT plugin; register with pluginlib; end-to-end sim test

Weeks**Milestone**

- 9–10 Generate synthetic dataset; run prompt evaluation; tune few-shot examples
- 11–12 Collect 200 real robot trials; train confidence calibrator
- 13–14 Adversarial testing; latency profiling; ablation study
- 15–16 Patent provisional draft + paper write-up + demo video

The single biggest technical risk is inference latency — 800ms on Jetson Orin is borderline acceptable for interactive use. Mitigate early: benchmark Q4_K_M vs Q2_K vs Q8_0 quantization in week 5, and implement a speculative fast-path where high-confidence alias exact-matches skip LLM inference entirely (string edit distance against annotation aliases, resolved in <1ms).